

Anatolia AI

Proje Dokümantasyonu

Katılım bankacılığı kampanya metinlerinden finansal bilgi çıkarımı, bankalar arası karşılaştırma ve doğal dil arayüzü. Şartnamenin on dokümantasyon kalemi, ölçülmüş sayılar ve açık eksiklerle.

Takım	Anatolia AI
Ekip	Mehmet Efe Aytaş (kaptan) · Irmak Altay · Ayça Engindeniz · Ecegüneş Dağ
Belge tarihi	22 Ağustos 2026
Korpus	2.708 belge · 11 kaynak · 7.022 çıkarılmış alan
Kod durumu	3.552 test yeşil · 0 başarısız · commit 03822c24
Ölçüm	yapılandırılmış mikro-F1 0,823 · halüsinasyon 0,034
Lisans	Apache-2.0

Her sayı bir komut çıktısına ya da artefakt dosyasına referans verir. Ölçülmemiş kalemler **Bölüm 10**'un sonunda ayrı listede toplanır.

İçindekiler

1. Sistem mimarisi ve veri akışı

2. NLP yaklaşımı

3. Veri seti

4. Ön işleme adımları

5. Model ve kural yapısı

6. Bankalar arası karşılaştırma

7. Kurulum ve çalıştırma

8. Karşılaşılan problemler ve çözümler

9. Model çıktı örnekleri

10. Başarım değerlendirme yöntemi

Kaynaklar

1. Sistem mimarisi ve veri akışı

Sistem tek bir orkestrasyon fonksiyonunda toplanır: `src/pipeline.py` `run_pipeline()`. On bir aşama var; her biri bağımsız test edilen bir modül.

```
toplama → temizlik → ön işleme → sınıflandırma → kural çıkarımı
→ (LLM katmanı: kodda var, üretimde kapalı) → uzlaştırma → normalizasyon
→ çelişki tespiti → depolama → karşılaştırma → panel + sohbet
```

#	Aşama	Modül	Giriş noktası
1	Veri toplama	<code>src/scraping/</code>	<code>collector.py</code> <code>collect_live()</code> , <code>collect_from_fixtures()</code>
2	Ön işleme	<code>src/preprocessing/clean.py</code>	<code>normalize_text()</code> , <code>tr_fold()</code>
3	Kampanya türü sınıflandırma	<code>src/extraction/ner/classifier.py</code>	<code>RuleHintClassifier</code>
4	Kural çıkarımı, 12 alan	<code>src/extraction/rules/extract.py</code>	<code>extract_all()</code>
5	LLM katmanı	<code>src/extraction/llm/extractor.py</code>	<code>LLMExtractor.extract()</code>
6	Uzlaştırma	<code>src/extraction/reconcile.py</code>	<code>reconcile()</code>
7	Normalizasyon	<code>src/normalization/normalize.py</code>	<code>normalize_rate()</code> , <code>normalize_money()</code>
8	Çelişki tespiti	<code>src/comparison/contradiction.py</code>	<code>detect()</code> , <code>detect_across()</code>
9	Depolama	<code>src/db/factory.py</code>	<code>create_repository()</code>
10	Karşılaştırma	<code>src/comparison/compare.py</code>	<code>rank()</code> , <code>rank_advantageous_by_type()</code>
11	Sunum	<code>src/api/main.py</code> + <code>web/</code>	<code>build_app()</code>

Dört çalışma modu

`MODE_FIXTURE` üç sabit belgeyle saniyenin altında koşar ve birim testleri besler. `MODE_CORPUS` tam korpusu tarar; teslim edilen demo veritabanını bu mod üretir. `MODE_LIVE` banka sitelerinden canlı toplar. `MODE_AUTO` eldeki veriye bakıp birini seçer. Aynı kod yolu dört modda da koşar, yani hızlı testle teslim koşumu arasında davranış farkı kalmaz.

İki veritabanı arka ucu

`src/db/base.py` içindeki `RepositoryProtocol` on üç metotlu tek bir sözleşme tanımlar. SQLite ve PostgreSQL/pgvector bu sözleşmeyi ayrı ayrı uygular; `create_repository()` `DATABASE_URL` doluysa Postgres'i, boşsa SQLite'i seçer.

Sözleşmenin bir bedeli vardı ve ödendi: depo katmanı dışında SQL yazmak yasaklandı. Eskiden bir `rows()` kaçış kapısı duruyordu ve beş yerden ham SQL çağırılıyordu. SQLite `?`, PostgreSQL `%s` yer tutucusu kullandığı için o beş çağrı Postgres'te `ProgrammingError` ile düşüyordu; soyutlama kâğıt üstünde kalmıştı. Beş sorgu sözleşme metotlarına taşındı, kaçış kapısı kapatıldı.

Kurumun kendi PostgreSQL'ine takılabilmek şartname §7'nin "kurum sistemlerine entegre edilebilir mimari" maddesinin doğrudan karşılığı. Parite ölçüldü: `test_repo_parity.py` 26 test, `test_pgvector_repository.py` 27 test.

2. NLP yaklaşımı

Üç katman, açık öncelik

`reconcile.py` içindeki `_PRIORITY` sabiti sırayı belirler: `**kural (3) > ner (2)`

İlm (1)**. Çoğu proje LLM'i merkeze koyar; burada tersi geçerli ve gerekçesi

bir bütçe hesabı.

Anotasyon bütçesi 150–300 örnek. Bu bütçe NER ince ayarına harcanırsa 12 alan × 8 tür uzayında ezberleme riski büyük. Aynı bütçe gold/eval setine harcanırsa rubriğin %30'luk model başarımı kalemi ölçülebilir hâle gelir. İnce ayar bütçesi yalnız 8 sınıflı kampanya türü sınıflandırıcısına ayrıldı; orada dengeli 150–300 örnek yeterli olurdu. BERTurk ince ayarı koştı ve kabul kapısında kaldı (makro-F1 0,565, kural ipucu katmanı 0,762). Üretimde sınıflandırmayı `RuleHintClassifier` yapıyor. Karar kaydı: `../decisions/ner-fine-tune-yerine-kural-few-shot.md`, ölçüm: `docs/rapor/berturk-ince-ayar-plani.md`.

Kurallar neden birincil

Katılım bankacılığı metninde çıkarılacak alanların çoğu sayısal ve yapısal: oran, tutar, vade, taksit, ücret, tarih. Deterministik regex artı normalizasyon bu alanlarda dört somut avantaj verir.

- Değer uyduramaz. Kural bulamazsa `null` döner.
- Kaynak konumunu kesin verir. `span_start` / `span_end` doğrudan eşleşme konumundan gelir.
- Hızlı. Belge başına p50 = 1,03 ms.
- Tekrar üretilebilir. Sıcaklık, tohum ve model sürümü değişkeni yok.

LLM katmanı: yalnız kural boşluklarını doldurur

LLM yalnız kuralların kaçırdığı örtük ifadeler için tasarlandı: "ilk 3 ay ödemesiz", "avantajlı kâr payı fırsatı", dolaylı oran anlatımları. Kısıtlı JSON çözümlemesi zorunlu; serbest metin hiç ayrıştırılmaz (`llm/schema.py` `guided_json_schema()`).

Üretim yolundan çıkarma kararını ölçüm dayattı. 20 Ağustos 2026, `gold.v2` (48 kayıt, 40'ı zor), Ollama artı `qwen2.5:7b-instruct`, CPU, `LLM_STRICT=1` :

kol	mikro-F1 (<code>strict</code>)	halüsinasyon	McNemar, <code>kural</code> karşısında
kural	0,4771	0,0425	—
llm	0,2545	0,0582	p = 0,00105
hibrit	0,4402	0,1029	p = 0,00050
hibrit-verify	0,3672	0,0984	p = 0,0000123

LLM sağlığı temiz: 384 çağrının 384'ü başarılı, parse hatası yok, şema ihlali yok. Yani düşük başarımın kaynağı teknik bir arıza olamaz. Model çağrıldı, geçerli JSON döndürdü ve yine kaybetti. Hibrit kol halüsinasyonu 2,4 katına çıkardı.

Mekanizma yapısal. `reconcile.py` sözleşmesi LLM'e kuralın yanlış pozitiflerini düzeltme yetkisi vermez; yalnız kuralın boş bıraktığı alanlara değer ekleyebilir. Kazanç tavanı dar, kayıp tabanı geniş.

Tablodaki `kural` sayısı (0,4771) o günün ağacına ait. Aynı gün iki tur kural iyileştirmesi koştı ve manşet 0,5702'ye çıktı; ablasyon o iyileştirmelerden önceki kesiti ölçüyor. Ayrıntı: `docs/rapor/ablasyon.md` .

Bu tablo 20 Ağustos'ta LLM'i çıkarımın BİRİNCİL kolu olmaktan çıkardı; 24 Ağustos'ta ise dar bir boşluk-doldurma rolüyle geri açtık. Fark yetkide: LLM artık kuralla yarışmıyor, yalnız kuralın hiç değer üretmediği alanlara aday veriyor ve her aday iki kapıdan geçmek zorunda —

- 1. dayanak kapısı:** çıkarılan her sayı belge metninde birebir geçmeli, yoksa aday düşer (`llm/dayanak.py` , `metinde_dogrula`);
- 2. alan kapısı:** kanıt penceresi alanın anlamıyla çelişmemeli — bir `kar_payi_orani` adayının penceresinde "komisyon", "BSMV", "stopaj" gibi izler varsa aday düşer (`YASAK_IZLER`).

Ölçülen sonuç: 120 belgelik denetimde **0 yanlış pozitif**; korpus genelinde **27 alan** kabul edildi (18 `kar_payi_orani` , 9 `finansman_tutari`). Dolayısıyla teslim edilen korpusun katman kırılımı **7.022 rule + 27 llm = 7.049**'dur. Oran değişmiyor: alanların **%99,6'sı** hâlâ kural üretimi.

Sohbet arayüzü: iki yol, bir yönlendirici

Sohbet katmanı saf RAG değil. `chatbot/router.py` `route()` soruyu sınıflandırır. Sayısal ve karşılaştırmalı sorular ("en düşük kâr payı hangi bankada") `extracted_fields` tablosu üzerinde yapısal sorguya gider. Koşul ve açıklama soruları ("konut finansmanı kampanyasının koşulları neler") RAG koluna gider. Semantik benzerlik araması sıralama sorularına yapısı gereği zayıf cevap verir; "en düşük" ifadesi bir vektör uzayında sıralama yapmaz. Hangi yolun koştığı cevabın yanında etiket olarak görünür.

Bu yönlendirme alan çıkarımından bağımsız bir mekanizma. Ablasyon tablolarındaki "hibrit" sözcüğüyle karıştırılmaması gerekir.

Güven skoru ve kalibrasyon sınırı

Kaynak	Hesap	Etiket
Kural	taban 0,70 artı tetikleyici kelimeye mesafe düzeltmesi	rule_heuristic
LLM	$\exp(\text{mean}(\text{logprob}))$, değer in kapsadığı jetonlar üzerinden	logprob

Bu skorlar kalibre edilmedi. "0,90 güven" %90 doğruluk anlamına gelmez; skor bir sıralayıcı olarak iş görür ve aynı alan içinde hangi çıkarımın daha güvenilir olduğunu söyler. Her alan `confidence_source` taşır, böylece ayırım panelde ve denetimde görünür kalır. Teslim edilen 7.049 alanın 7.022'si `rule_heuristic` , 27'si `logprob` taşıyor.

3. Veri seti

Kapsam

Kapsamı BDDK'nın katılım bankaları listesi belirler. `config/banks.yaml` on bankayı yapılandırma olarak tutar; yeni banka eklemek tek YAML bloğu, kod değişmez. TKBB (Türkiye Katılım Bankaları Birliği) terminoloji kaynağı olarak ayrıca toplanır.

Kaynak	Belge
Kuveyt Türk	885
Albaraka Türk	353
Vakıf Katılım	307
Ziraat Katılım	276
T.O.M. Katılım	260
Türkiye Emlak Katılım	227
Türkiye Finans	205
Dünya Katılım	110
Hayat Finans	72
Adil Katılım	11
TKBB	2
Toplam	2.708

Belge türü kırılımı: 1.726 kampanya sayfası, 982 sözleşme ve tarife belgesi (çoğu PDF'ten dönüştürülmüş).

Durum kırılımı: 434 belge süresi dolmuş kampanya olarak işaretli.

Dengesizlik açıkta duruyor. Adil Katılım 11, TKBB 2 belge veriyor; bu kurumların siteleri küçük ya da yapısı farklı. Dengesizliği raporluyoruz çünkü karşılaştırma sonucunu doğrudan etkiliyor: az belgeli bankanın "en iyi" çıkma olasılığı yapısal olarak düşük.

Toplama yöntemi ve etik sınırlar

`src/scraping/robots.py` `robots.txt` dosyasını ayrıştırır ve ona uyar; engellenen yollar `_products_report.json` içinde `blocked` listesine yazılır. `Ratelimiter` alan adı başına varsayılan 3 saniye bekler. User-Agent açıklayıcı: `AnatoliaAI-Research/1.0`. JavaScript ile üretilen sayfalar için Playwright, statik sayfalar için requests artı BeautifulSoup koşar.

Toplama dört turda yapıldı: aktif kampanyalar, ürün sayfaları (oran ve vade taşıyanlar), süresi dolmuş kampanya arşivi, PDF ücret tarifeleri ve sözleşme formları.

Köken kaydı

Her belge yanında bir `.txt.meta.json` dosyası taşır: `source_url`, `scraped_at`, `content_hash`, `collection_method`. Ölçülen doluluk: `source_url` 2.708/2.708, `scraped_at` 2.706/2.708 (eksik ikisi demo fıkstürü).

Zincir kesintisiz: alan → karakter konumu → belge → kaynak URL → toplama zamanı → içerik hash'i. Panelin denetim sekmesi bu zinciri uçtan uca gösterir.

Kampanya türü dağılımı

Tür	Belge
Kart	901
Finansman	592
Yatırım Ürünü	526
Konut Finansmanı	118
İhtiyaç Finansmanı	112
Taşıt Finansmanı	68
Alışveriş Puanı	61
Yeni Müşteri	5
<i>sınıflanamayan</i>	325

Şartname §5.4'ün saydığı sekiz türün tamamı üretiliyor. 325 belge (%12,0) hiçbir türe girmiyor ve ayrı satır olarak raporlanıyor; zorlama etiket atanmıyor.

Alan kapsamı ve en büyük kısıt

Alan	Belge	Pay
kampanya_kosullari	2.301	%85,0
kampanya_suresi	1.179	%43,5
masraf_durumu	814	%30,1
hedef_kitle	746	%27,5
vade_ay	627	%23,2
taksit_sayisi	426	%15,7
odul_miktari	331	%12,2
alisveris_puani	182	%6,7
kar_payi_orani	164	%6,1
indirim_orani	162	%6,0
finansman_tutari	84	%3,1
tahsis_ucreti	33	%1,2
Toplam alan	7.049	

Son satırdan bir tanesi projenin en önemli bulgusu. **Kâr payı oranı yalnız 164 belgede geçiyor.** Şartname §5.7'nin birinci karşılaştırma ölçütü ("En Düşük Kâr Payı Oranı") tam bu alana dayanıyor.

Sebebi veri kaynağının doğasında. Katılım bankaları kampanya sayfalarında oranı çoğu zaman yazmıyor; "avantajlı oran", "özel oranlı finansman" gibi nitel ifadeler kullanıyor. Sistem bu boşluğu doldurmuyor, `comparable=False` ile işaretliyor. PDF hasadı payı 60'tan 146'ya, LLM boşluk doldurma koşumu 146'dan 164'e çıkardı; alan yine de seyrek.

Asıl çözüm başka yerden geldi: **oranı bankalar yayımlamıyor ama TKBB yayımlıyor.** Ayrıntı aşağıda, "Katılma hesabı oranları" başlığında.

Katılma hesabı oranları — korpus dışı ikinci kaynak

Yukarıdaki kısıt (kâr payı oranı belgelerin yalnız %6,1'inde geçiyor) kampanya korpusundan çözülemez, çünkü bilgi orada YOK. Bankalar katılma hesabı oranını HTML kampanya sayfasında değil, TKBB'nin ortak yayınında açıklıyor. Bu yüzden korpusun yanına ayrı bir veri kolu eklendi.

İki ayrı uç noktası hasat edildi:

Kaynak	Kapsam	Kayıt
TKBB Veri Peteği (cari hafta)	9 banka × 4 rapor × 4 vade × 4 para birimi	245
TKBB tarihsel arşiv (2012–2025)	aynı kırılım, haftalık seri	210.474
Kuveyt Türk kendi yayını (PDF)	7 bakiye segmenti × vade	144

Üretim yolunda **yalnız cari hafta** okunuyor. Tarihsel arşiv depoda gzip'li duruyor (89 MB → 1,3 MB) ama hiçbir kod yolu onu açmıyor: trend analizi bu teslimin kapsamı dışında ve kullanılmayan bir veriyi "kullanıyoruz" gibi göstermek ölçüm dürüstlüğüne aykırı olurdu.

İki büyüklük asla aynı sıralamada yarışmaz. TKBB iki ayrı sayı yayımlıyor ve ikisi de "kâr payı oranı" diye anılıyor:

- **Dağıtılan kâr payı oranı** — gerçekleşen yıllık getiri (ör. %42,79).
- **Kâr paylaşım oranı** — kârın katılımcıya düşen payı (ör. %90).

Birini ötekiyle sıralamak, %90'lık bir bölüşüm oranını %42,79'luk bir getirinin üstüne koymak demektir; sonuç sayısal olarak "doğru" görünür ve tamamen anlamsızdır. Sistem her kaydı **buyukluk** alanıyla (**getiri** / **pay**) etiketliyor ve iki kümeyi ayrı tablolarda gösteriyor.

Doğrulama. Albaraka'nın TL paylaşım oranları (90 / 90 / 92 / 93) üç bağımsız kaynakta birebir aynı çıktı: cari uç nokta, tarihsel arşiv ve bankanın kendi yayımladığı PDF.

Bilinen boşluklar. ① Haziran 2025 – Ağustos 2026 arası hiçbir uçta yok. ② Segment kırılımı yalnız Kuveyt Türk'te tam (144 kayıt); Emlak'ta iki segment var, yedi bankada hiç yok — bankalar yayımlamıyor. ③ 2026 yılı tarihsel arşivde yok, çünkü yıl bitmedi; arşiv yıl kapandığında dolar.

Yüzey: `GET /katilma-oranlari` ucu, panelin *Katılma Oranları* sekmesi ve sohbetin `katilma_orani` yolu. Hasat betikleri: `scripts/tkbb_guncel_hasat.py` , `scripts/tkbb_karpayi_hasat.py` , `scripts/kt_paylasim_pdf.py` .

TLS notu. Tarihsel arşiv ucunun sertifikası süresi dolmuş durumda. Betik doğrulamayı **sessizce atlamıyor**: `--sertifika-atla` bayrağı verilmezse 2 koduyla çıkıyor ve gerekçeyi yazıyor. Bayrakla toplanan her kayıt `tls_dogrulama: "atlandi"` damgası taşıyor. İçerik ayrıca kullanıcının tarayıcı çıktısıyla karşılaştırıldı: 7 bankada 7'si birebir tuttu.

Bilinen veri kalitesi sorunları

Sorun	Ölçülen büyüklük	Durum
Menü ve gezinme metni kaynak penceresine sızıyor	sohbet kaynak tablosunda gözle görülür	açık
Pazarlama metnindeki bağlamsız %0 oranlar sıralamaya giriyor	Kuveyt Türk API market sayfası vb.	açık
Okunamaz PDF (ToUnicode tablosu olmayan gömülü yazı tipi) çöp metni koşul kalemi olarak sunuyordu	1 Albaraka sözleşmesi	kapatıldı, <code>_ortak.bozuk_metin</code> kapısı
İkili çöp belge: 352 NUL baytı, 0 alan	1 belge	kapatıldı, <code>NulByteInText</code>

4. Ön işleme adımları

`src/preprocessing/clean.py` yedi fonksiyon taşır.

Fonksiyon	İş
<code>strip_html()</code>	BeautifulSoup ile etiket temizliği; script ve style atılır
<code>normalize_text()</code>	boşluk sıkıştırma, tekrarlanan satır kaldırma
<code>split_sentences()</code>	Türkçe cümle bölme, kısaltma listesi farkında
<code>tr_fold()</code>	Türkçe-doğru küçük harf
<code>tr_upper()</code>	Türkçe-doğru büyük harf
<code>tr_fold_ascii()</code>	ASCII sadeleştirme, eşleşme için
<code>slugify_tr()</code>	dosya adı üretimi

`tr_fold()` neden ayrı bir fonksiyon

Python'un `str.lower()` Türkçe için yanlış sonuç verir. `'İ'.lower()` kombine noktalı `i` üretir, `'I'.lower()` ise `'i'` verir; Türkçede `'ı'` olması gerekir. Sonuç:

```
'ÜCRETSİZ'.lower() → 'ücretsiz' # 'ücretsiz' ile eşleşmiyor
```

Bu tek satırlık fark `masraf_durumu` alanını ters çeviriyordu: büyük harfle yazılmış "ÜCRETSİZ" tanınmayınca sistem "masraf var" diyordu. Aynı tuzak projede üç ayrı yerde hata üretti — masraf çıkarımı, güvenlik kapısının terim eşleşmesi, sınıflandırıcı ipuçları.

Bunun üzerine P2 ortografik değişmezliği yazıldı: `çıkar(metin) == çıkar(BÜYÜK(metin))`. Denetim ilk koşuda bu sınıftan 104 ihlal buldu.

bs4 sapması

Teslim imajının ince bağımlılık listesi (`requirements-api.txt`) BeautifulSoup taşımıyordu. `strip_html()` bs4 yokken sessizce ham HTML'e düşüyordu.

Ortam	Belge başına ortalama karakter
Geliştirme, bs4 var	4.317
Teslim imajı, bs4 yok	6.232
Düzeltilmeden sonra	4.320

Teslim imajı %44 gürültüyle koşuyordu. Birim testler bs4'ün kurulu olduğu geliştirme ortamında koştuğu için hatayı görmedi, çünkü fonksiyon istisna fırlatmıyordu; sessizce kötü sonuç veriyordu. İmaj 406 KB büyüdü, gürültü kalmadı.

5. Model ve kural yapısı

Her alan kendi kanıtını taşır

`src/schemas.py` `ExtractedField`:

Alan	İçerik
<code>raw_value</code>	metindeki hâli, <code>"%1,89"</code>
<code>canonical_value</code>	normalize edilmiş hâli, <code>1.89</code>
<code>confidence</code>	0–1 skor
<code>confidence_source</code>	<code>rule_heuristic</code> <code>logprob</code> <code>self_reported</code>
<code>source_span</code>	±40 karakterlik bağlam penceresi
<code>span_start</code> / <code>span_end</code>	kesin karakter konumu
<code>extractor</code>	<code>rule</code> <code>ner</code> <code>llm</code>

`verify_span()` öz-denetim yapar: konumla kesilen metin gerçekten `raw_value` içeriyor mu? Teslim edilen korpusta 7.049 alanın 7.049'unda konum dolu ve doğrulanmış — LLM'in doldurduğu 27 alan dâhil, çünkü dayanak kapısı zaten metinde birebir geçen bir dizi arıyor.

Bu konumlar bir dönem veritabanı sınırında kayboluyordu. API katmanı `str.find()` ile yeniden aramak zorunda kalıyor, aynı değer metinde iki kez geçtiğinde yanlış yeri gösteriyordu. Şimdi saklanan konum birincil, `str.find()` yalnız yedek.

On iki alan ve çıkarıcıları

#	Alan	Kanonik tip	Çıkarıcı (rules/extract.py)
1	kar_payi_orani	number {min,max}	extract_kar_payi() , ileri kalıp, oran tablosu
2	finansman_tutari	{value, currency:"TRY"}	extract_tutar()
3	vade_ay	integer	extract_vade()
4	taksit_sayisi	integer	extract_taksit()
5	tahsis_ucreti	{value, currency}	extract_tahsis_ucreti()
6	masraf_durumu	{has_fee, amount}	extract_masraf()
7	odul_miktari	{value, currency}	extract_odul_miktari()
8	indirim_orani	number {min,max}	extract_indirim_orani()
9	alisveris_puani	{kind, value}	extract_alisveris_puani()
10	kampanya_suresi	ISO-8601 string	extract_kampanya_suresi()
11	kampanya_kosullari	array[string] , en çok 12	extract_kampanya_kosullari()
12	hedef_kitle	array[enum] , en çok 4	extract_hedef_kitle()

Kampanya türü şartname tablosunda on üçüncü alan olarak sayılır. Teknik olarak bir sınıflandırma çıktısı ve `campaigns.campaign_type` sütununda durur; bu yüzden çıkarım şeması 12, şartname tablosu 13 sayıyor. İki farklı payda, çelişki yok.

Normalizasyon

Şartname §5.6'nın istediği tekilleştirmeyi `src/normalization/normalize.py` yapar.

Girdi	Çıktı
%2,05 · % 2.05 · 2.05 %	2.05
500 TL · 500₺ · 500 Türk Lirası	{value: 500, currency: "TRY"}
12 ay · 1 yıl	12
31.12.2026 · 31 Aralık 2026	2026-12-31
1.500,00	1500.00
masrafsız · ücretsiz · dosya masrafı yok	{has_fee: false, amount: 0}

Dejenere aralıklar kanonik biçime indirgenir: `{min: 1.89, max: 1.89} → 1.89` (`collapse_degenerate_range`). İndirgeme olmadan bu değer `comparable=False` sayılıyor ve sıralamadan sessizce düşüyordu; en iyi teklif kaybediliyordu.

Zor anlama vakaları

Beş vaka sınıfı açıkça ele alınır: değer aralıkları, birim karışması, negasyon, örtük hedef kitle, bilgi yokluğu. Örnek girdi:

"İhtiyaç finansmanında kâr payı oranı %1,99 - %2,49 arasında, 36 aya kadar vade. İlk 3 ay ödemesiz seçeneği ile 24 taksit. Yeni müşterilerimize dosya masrafı yok."

Vaka	Sonuç
Aralık	%1,99 – %2,49 aralık olarak korundu, ortalamaya indirgenmedi
Birim karışması	36 ay vade olarak alındı, oran sanılmadı
Negasyon	"dosya masrafı yok" → tahsis ücreti 0 TL, <code>null</code> değil
Hedef kitle	"Yeni müşterilerimize" → <code>yeni_musteri</code>
Bilgi yokluğu	6 alan boş bırakıldı, panel bunu açıkça yazıyor

Açık kalan kısım: "İlk 3 ay ödemesiz" gibi zaman-koşullu ifadeler ayrı alan olarak çıkarılmıyor. Bu vaka sınıfı LLM katmanının işi, LLM ise üretimde kapalı.

Sözcük sınırı zorunluluğu

Sınıflandırıcı bir dönem alt-dize eşleşmesi kullanıyordu. `"ev"` ipucu `"devam"`, `"seviye"`, `"evrak"` kelimelerinin içinde eşleşiyordu. Sonuç: korpusun %48'i sahte "Konut Finansmanı" etiketi alıyordu. `synonyms.py` `keyword_pattern()` artık zorunlu sözcük sınırı (`\b`) uygular.

Bu hata hiçbir testi kırmadı, hiçbir istisna fırlatmadı. Yarım korpusu yanlış etiketledi ve panel bunu güvenle gösterdi. En pahalı hatalar çökmüyor.

6. Bankalar arası karşılaştırma

Beş ölçüt

Ölçüt	Uygulama
En Düşük Kâr Payı Oranı	<code>compare.py</code> <code>rank()</code> , <code>lower_is_better</code>
En Yüksek Ödül Miktarı	<code>rank()</code> , <code>higher_is_better</code>
En Uzun Vade Seçeneği	<code>rank()</code> , <code>higher_is_better</code>
En Düşük Masraf	<code>rank()</code> , masraf <code>amount</code> üzerinden
En Avantajlı Kampanya	<code>rank_advantageous_by_type()</code> , tür içinde bileşik skor

Adil kıyas garantisi

Karşılaştırmada en kolay yapılan hata eksik alanı 0 puan saymak. Bu, veri toplanamamış bankayı "en kötü" gösterir ve sıralamayı sessizce yanıltır. Sistem dört kapı koyar.

- Eksik alan 0 sayılmıyor; satır `comparable=False` işaretlenip sebebi `note` alanına yazılıyor.
- `MIN_COVERAGE = 0.5` — alan kapsamı %50'nin altındaki kampanya bileşik skora girmiyor.
- Aralık değerler doğrudan kıyaslanmıyor; `comparable=False` oluyor ama gizlenmiyor, notuyla listenin sonuna geçiyor.
- `MIN_GROUP_SIZE = 3` — üçten az kampanyası olan tür sıralanmıyor, listede görünmeye devam ediyor.

Bileşik skor ağırlıkları

"En Avantajlı Kampanya" tek alandan çıkmaz. `DEFAULT_WEIGHTS` ve `WEIGHT_RATIONALE` birlikte kayıtlı:

Alan	Ağırlık	Gerekeçe
Kâr payı oranı	0,40	toplam maliyeti en çok belirleyen kalem
Masraf durumu	0,20	doğrudan nakit çıkışı
Ödül miktarı	0,15	tek seferlik kazanç
Vade	0,15	esneklik göstergesi
Finansman tutarı	0,10	erişilebilirlik göstergesi

Normalizasyon sıralama tabanlı. Min-max normalizasyon tek aykırı değere aşırı duyarlı; %50 kâr payı taşıyan bir belge tüm ölçeği bozar. Sıralama tabanlı yaklaşım bundan etkilenmiyor.

Sıralama tür içinde yapılıyor. Konut finansmanı ile alışveriş puanı kampanyasını aynı ölçekte sıralamak anlamsız sonuç üretir.

Skorlama şeffaflığı

`GET /scoring?field=...` formülü, adımları ve ağırlık manifestosunu döndürür; panelin `ScoringExplainer` bileşeni bunu gösterir. "Bu sıralama nasıl çıktı" sorusunun cevabı bir API çıktısı.

Tek alanlı sıralamalarda `composite_weights: null` döner ve şu not eklenir: kod tabanında alanlar arası ağırlıklı bileşik skor yok, sıralama tek alan üzerinden yapılıyor.

Çelişki tespiti

`src/comparison/contradiction.py` yedi çelişki türü tanımlar. Bugünkü korpusta üçü tetikleniyor.

21 Ağustos 2026, tam korpus taraması (`python -m src.comparison.scan`):

tür	adet	kırılım
belgeler arası	8	6 çapraz bitiş tarihi, 2 çapraz kâr payı uyumsuzluğu
belge içi	20	17 süresi dolmuş kampanya, 2 çelişen tutar bandı, 1 çelişen bitiş
toplam	28	

Manşet örnek: Albaraka Türk aynı ürün için iki ayrı formda %7,0 ve %1,0 kâr payı oranı yayımlamış. Kesişmeyen iki oranı `detect_across()` üretim koduyla yakaladı.

Çelişki sayısı hangi kod yolunun koştuğuna bağlı ve bu ayırım kayıtlı. Zaman bağımlı kural (`suresi_dolmus_kampanya`) yalnız `as_of` verildiğinde koşar. `run_pipeline` bunu geçmiyor, API geçiyor:

yol	as_of	çelişki	tür
<code>run_pipeline(mode="corpus")</code> – CLI	yok	3	2
<code>GET /contradictions</code> – panelin gösterdiği	var	20	3

Yanlış negatif oranı ölçülmedi. 2.708 belgede 28 çelişki, mekanizmanın çalıştığını gösterir; kuralların ne kadar muhafazakâr olduğunu göstermez.

7. Kurulum ve çalıştırma

Python 3.11 veya üstü gerekir. Kod `zip(..., strict=)` gibi 3.10+ sözdizimi kullanıyor. Stok macOS `/usr/bin/python3` sürümü 3.9.6 ve orada komutlar `TypeError` ile düşer.

En kısa yol

```
cd app
make baslat      # API + panel + yerel model, port sahipliği denetimiyle
make durdur      # modeli bellekten indirir, süreçleri toplar
```

`scripts/baslat.sh` altı ortam değişkenini kendisi kuruyor ve bitişte ölçülmüş durum çıktısı basıyor. Betik, elle başlatmanın somut bir arızasından doğdu: 8000 portunu LLM'siz açılmış eski bir `uvicorn` tutuyordu, panel doğru davranıp "yerel model kapalı" diyordu, model ise başka bir portta açıktı.

Elle kurulum

Veritabanı önce kurulur, sunucu sonra açılır.

```

cd app
python3 -m venv .venv && ./venv/bin/pip install -r requirements.txt

# 1) Demo veritabanını üret (~55 s, ölçüm 2026-08-21 arm64)
python3 -m scripts.build_demo_db --out data/demo.db

# 2) Özetleri geri yükle – bu adımı atlamayın
python3 -m scripts.ozet_geri_yukle --db data/demo.db

# 3) API
DATABASE_PATH=data/demo.db LLM_BACKEND= \
  ./venv/bin/python -m uvicorn src.api.main:app --port 8000

# 4) Panel
cd web && npm install && NEXT_PUBLIC_API_URL=http://127.0.0.1:8000 npm run dev

```

İki uyarı, ikisi de sessiz düşüş üretir.

`DATABASE_PATH` verilmezse varsayılan `:memory:` devreye girer, `/stats` `campaigns: 3` döner ve panel 2.708 belge yerine üç fıkstür gösterir. Hata mesajı çıkmaz.

`build_demo_db` `campaigns.ozet` sütununu bilmiyor ve boş bırakıyor. İkinci adım atlanırsa panel her belgede "AI Özeti üretilmedi" yazar. Özetler yerel modelle üretildi (~4 saat) ve `data/ozet-yedeği.json` içinde git'te izleniyor; geri yükleme saniyeler sürüyor ve model istemiyor. Kural tabanlı sahte özet basmak kodda yasak (`src/summarize/ozet.py`), bu yüzden yedek tek meşru yol. Ölçülen kapsam: 2.708 belgenin 2.634'ü özetli (%97,3); kalan 74'ün sebebi boş metin ya da terminoloji kapısının reddi.

Docker

```

cd app
python3 -m scripts.build_demo_db --out data/demo.db # derlemeden ÖNCE
python3 -m scripts.ozet_geri_yukle --db data/demo.db
docker compose up
# api : http://localhost:8000 (SQLite, 2.708 belge, LLM kapalı)
# web : http://localhost:3000 (panel + sohbet)

```

`Dockerfile.api` `data/` dizinini derleme anında imaja kopyalıyor. `*.db` `.gitignore` içinde olduğu için temiz bir klonda o dosya yok; sıralama bozulursa API fıkstürlere düşüyor.

Diğer profiller:

```

docker compose --profile postgres up # postgres + api-postgres(8010) + pgvector
docker compose --profile gpu up # vLLM (8001)
docker compose --profile ollama up # Ollama (11434), CPU yedeği

```

Tüm imajlar `@sha256:` digest ile pinli.

Doğrulama

```
curl -s localhost:8000/health # {"status":"ok","llm":false,"backend":"sqlite"}
curl -s localhost:8000/stats # campaigns: 2708, banks_with_campaigns: 11, fields: 7022
```

Değerlendirme ve test komutları

```
python3 -m unittest discover -s tests # 3.948 test toplanır
python -m eval.properties --raw-dir data/raw # değişmez denetimi
python -m eval.run_eval --gold data/gold/gold.v2.json --config kural
python -m eval.ablation # kural / llm / hibrit / hibrit-verify
python -m src.comparison.scan --raw-dir data/raw # çelişki taraması
python -m scripts.kanit_tazeligi # yayımlanan sayı ↔ artefakt karşılaştırması
bash scripts/offline_proof.sh # ağ izolasyonu kanıtı
ruff check .
```

Kanonik test koşucusu `unittest` . `pytest` bağımlılık listesinden düşürüldü; hiçbir test dosyası `pytest` API'si kullanmıyor. Çapraz doğrulama isteyen `pip install pytest` ile aynı sayıyı üretir.

Ortam değişkenleri

Değişken	Varsayılan	Etki
<code>DATABASE_URL</code>	boş	dolu → PostgreSQL, boş → SQLite
<code>DATABASE_PATH</code>	<code>data/demo.db</code>	SQLite dosyası; <code>:memory:</code> üç fıkstüre düşer
<code>LLM_BACKEND</code>	boş	boş → <code>NullLLMExtractor</code> , tam offline
<code>LLM_STRICT</code>	boş	<code>1</code> → LLM erişilemezse sessiz düşme yasak
<code>RAG_RETRIEVER</code>	<code>keyword</code>	<code>keyword</code> <code>auto</code> <code>vector</code>

Canlı toplama için ek adım

Bankaların dördü (Adil Katılım, Hayat Finans, T.O.M., Türkiye Finans) sayfalarını JavaScript ile üretiyor. `pip install` yalnız Python paketini kuruyor, tarayıcı ikilisini indirmiyor:

```
python -m playwright install chromium # ~95 MB, internet gerektirir
```

Teslim edilen sistem bu adım olmadan da tam çalışır; demo önceden doldurulmuş veritabanından okuyor. Tarayıcı yalnız yeni veri toplarken gerekiyor ve eksikse sistem o bankayı Türkçe bir açıklamayla atlayıp diğerlerini sürdürüyor.

8. Karşılaşılan problemler ve çözümler

Ortak örüntü: aşağıdaki on bir hatanın hiçbirisi çökme üretmedi. Hepsi sessizce yanlış değer veriyordu. Bölüm 10'daki değişmez denetimi tam bu yüzden yazıldı.

P1 — Türkçe küçük harf

Kök neden: `str.lower()` `İ/I` çiftini yanlış çeviriyor. Etki: `masraf_durumu` ters çevriliyor; "ÜCRETSİZ" belge "masraf var" oluyor. Çözüm: `tr_fold()`, tüm eşleşmeler bundan geçiyor. Aynı tuzak üç ayrı yerde hata üretmişti. Doğrulama: P2 ortografik değişmezliği. Düzeltme geri alınınca denetim gerçekten ihlal veriyor.

P2 — Binlik ayırıcı

Kök neden: `float()` Türkçe sayı biçimini bilmiyor, `.` işaretini ondalık sanıyor. Etki: `"1.500,00 TL"` → `1,0 TL`. 1.500 liralık ücret 1 lira olarak kaydediliyor ve karşılaştırma tamamen bozuluyor. Çözüm: `parse_tr_number()`.

P3 — Hayali tahsis ücreti

Kök neden: ücret çıkarıcısının arama penceresi cümle sınırını aşıp sonraki cümledeki tarih sayısını yakalıyordu. Etki: `"ücret alınmaz. Kampanya 31 Aralık"` → `31 TL`. Kural katmanının kendi halüsinasyonu. Çözüm: pencere cümle sınırına bağlandı, negasyona öncelik verildi.

P4 — Sahte aralık

Kök neden: "ile" bağlacı aralık işareti sanılıyordu; vade sayısı oran aralığının üst sınırı oluyordu. Etki: `"%1,89 ile 120 aya kadar"` → `%1,89-%120`. Şartnamenin manşet örneği yanlış çıkarılıyordu. Çözüm: iki yönlü kâr payı çıkarımı, artı birim farkındalığı — `%` ile `ay` karışamaz.

P5 — Alt-dize eşleşmesi

Kök neden: `"ev"` ipucu `"devam"`, `"seviye"` içinde eşleşiyordu. Etki: korpusun %48'i sahte Konut Finansmanı etiketi alıyordu. Çözüm: `keyword_pattern()` zorunlu sözcük sınırı uyguluyor.

P6 — Terminoloji boşluğu

Kök neden: şartname §5.5'in beş kavramından ikisi ("finansman maliyeti", "katılım fonu") yalnız LLM istem metninde tanımlıydı; kural katmanı bu kavramları bilmiyordu. Etki: LLM kapalı teslim yapılandırmasında sistem 226 belgeyi kapsamıyordu. Çözüm: `TERMINOLOGY_TRIGGERS` kural katmanına taşındı.

P7 — bs4 eksikliği

Ayrıntı Bölüm 4'te. Teslim imajı %44 gürültüyle koşuyordu: 4.317 → 6.232 → 4.320 karakter/belge.

P8 — NUL baytı

Kök neden: PDF'ten dönüştürülmüş bir belge 352 adet `0x00` baytı taşıyordu. SQLite kabul ediyor, PostgreSQL `TEXT` sütununa yazarken hata veriyor. Etki: iki arka uç arasında sessiz veri farkı; parite iddiası çürüyordu. Çözüm: `NulByteInText` istisnası, artı `ON_NUL_MODES` politikası.

P9 — Banka filtresi halüsinasyonu

Kök neden: sohbetin RAG kolu banka adını filtrelemiyordu. Etki: "Ziraat Katılım'ın konut kâr payı" sorusu Kuveyt Türk'ün oranıyla cevaplanıyordu. Kullanıcıya doğru görünen, tamamen yanlış cevap. Çözüm: `detect_banks()`, `MIN_OVERLAP=2` kanıt eşiği ve `tr_fold` eşleşme.

P10 — `masraf_durumu` kesinliği

Kök neden: çıplak "tahsis ücreti" ifadesi ücret kanıtı sayılıyordu; başlıkta ya da gezinme şeridinde geçmesi yetiyordu. Çözüm: ücret iddiası için sayısal değer ya da açık negasyon zorunlu kılındı. Etki: 370 alan → 248 alan. Kapsam düştü, doğruluk arttı; bilinçli takas.

P11 — Okunamaz PDF

Kök neden: son PDF hasadındaki bir Albaraka sözleşmesi ToUnicode tablosu olmayan gömülü yazı tipi taşıyordu; dönüştürme çöp metin ürettiyordu. Etki: çöp metin `kampanya_kosullari` kalemi olarak sunuluyordu. Değişmez denetimi bunu iki gerçek ihlal olarak yakaladı ve CI'ı kırdı. Çözüm: kök neden kodda değil veride olduğu için çözüm bir kapı oldu — `_ortak.bozuk_metin`. İhlal gizlenmedi, sebebi kayıtlı.

Yan kayıt — SBOM sapması

Demo videosu seslendirmesinden kalan `edge-tts` ve yedi geçişli bağımlılığı `.venv` 'e sızmıştı; envanter 96'dan 105'e çıkmıştı. Ölçüm gösterdi ki SBOM o hâliyle tazelenirse CI lisans kapısı düşüyordu: `edge-tts` LGPL-3.0-only lisanslı ve kapının copyleft kovalasına giriyor. Dokuz paket kaldırıldı; ortam ile SBOM yeniden birebir örtüşüyor ve testler sıfır hatayla geçti.

9. Model çıktı örnekleri

Aşağıdaki satırlar `data/demo.db` içinden doğrudan okundu. Her biri gerçek bir belgeden, gerçek karakter konumuyla geliyor.

Sayısal oran, tek değer

```
{
  "banka": "Kuveyt Türk",
  "alan": "kar_payi_orani",
  "raw_value": "%4.19",
  "canonical_value": "4.19",
  "confidence": 0.85,
  "confidence_source": "rule_heuristic",
  "span": [94, 99],
  "source_span": "...Enerji Tasarrufu Haftasına Özel %4.19 Kar Oranı Fırsatı!...",
  "source_url": "https://www.kuveytturk.com.tr/kampanyalar/kampanya-arsivi/bisiklet-finansmaninda-..."
}
```

Oran tablosundan aralık çıkarımı

Sayfa vadeye göre dört ayrı oran veriyor. Sistem tabloyu tek bir aralığa indiriyor ve aralığı ortalamaya düşürmüyor.

```
{
  "banka": "Kuveyt Türk",
  "alan": "kar_payi_orani",
  "raw_value": "Vade Kar Oranı",
  "canonical_value": {"min": 3.23, "max": 3.54},
  "confidence": 0.9,
  "span": [1325, 1339],
  "source_span": "...Vade Kar Oranı 24 ay 3.54 36 ay 3.29 48 ay 3.23 60 ay..."
}
```

Negasyon: "ücretsiz" sıfır değeri üretir

```
{
  "alan": "masraf_durumu",
  "canonical_value": {"has_fee": false, "amount": 0.0},
  "confidence": 0.95,
  "span": [22, 114]
}
```

Bu örnekte bilinen bir kusur görünüyor: `raw_value` gezinme şeridini ("Ana Sayfa Kampanyalar Kampanya Arşivi") içine alıyor. Kanonik değer doğru, kaynak penceresi geniş. Açık kayıt, Bölüm 3'teki veri kalitesi tablosunda.

Tarih normalizasyonu

```
{
  "alan": "kampanya_suresi",
  "raw_value": "26.03.2026 - 31.05.2026",
  "canonical_value": "2026-05-31",
  "confidence": 0.72
}
```

Örtük hedef kitle

```
{
  "alan": "hedef_kitle",
  "raw_value": "müşterilerimize özel",
  "canonical_value": ["mevcut_musteri"],
  "confidence": 0.95,
  "source_span": "...Akaryakıt istasyonu ve marketi olan müşterilerimize özel yeni yapılan başvurularda..."
}
```

Koşul listesi

```
{
  "alan": "kampanya_kosullari",
  "canonical_value": [
    "Müşteri olma aşamasında \"Davet Kodu\" alanına \"KTOD2026\" kodunun yazılması gerekmektedir.",
    "Paketiniz kapsamında yer alan içeriklere erişmek için TOD'a üye olunması gereklidir.",
    "Paket kodları yalnızca yurt içi TOD servisi için geçerlidir.",
    "Paket kodları nakde çevrilemez, para iadesi yapılamaz ve devredilemez.",
    "Kampanyadan sınırlı sayıda müşteri yararlanabilir.",
    "Kampanya kazanımı müşteri bazlıdır ve her müşteri yalnızca bir kez faydalanabilir."
  ],
  "confidence": 0.85
}
```

Ödül miktarı

```
{
  "alan": "odul_miktari",
  "raw_value": "500 TL",
  "canonical_value": {"value": 500.0, "currency": "TRY"},
  "confidence": 0.95,
  "source_span": "...Kampanyadan maksimum kazanım tutarı 500 TL 'dir..."
}
```

Çelişki çıktısı

Girdi: *"Taşıt finansmanı kampanyası! Kâr payı oranı %2,49, 48 aya kadar vade. Tamamen masrafsız başvuru. Tahsis ücreti 1.500,00 TL olarak tahsil edilir."*

1.500,00 TL doğru okunuyor (1.500 TL) ve çelişki yakalanıyor: «masrafsız» belirtilmiş ancak tahsis ücreti var" — **masraf_durumu** ile **tahsis_ucreti** arasında.

Sohbet çıktıları

```
Soru : Kuveyt Türk konut finansmanı oranı ve vadesi ne?
Cevap: kâr payı oranı %1,89 · vade 120 ay
      (koşullu oran uyarısı ve katılma hesabı ihtarı ekli)

Soru : Kuveyt Türk mü avantajlı Ziraat Katılım mı?
Cevap: kâr payı / vade / masraf / ek ödül boyutlarında "çünkü ..." gerekçeli
      madde madde karşılaştırma, 7 kaynak satırı

Soru : XYZ Bankası konut finansmanı oranı ne?
Cevap: "Sorduğunuz bankayı veri setimde bulamadım" + tanınan 10 bankanın listesi
```

Üçüncü örnek çekimsizliği gösteriyor. Sistem tanımadığı bankaya karşılık başka bir bankanın verisini sunmuyor.

10. Başarım değerlendirme yöntemi

Ölçüm altyapısı

Modül	İçerik
<code>eval/run_eval.py</code>	alan bazlı P/R/F1, mikro ve makro, <code>fp_wrong</code> ayrı sayım, zor-vaka dilimi
<code>eval/matchers.py</code>	<code>strict_match</code> (tolerans 1e-9) ve <code>tolerant_match</code> (%1 bağlı)
<code>eval/stats.py</code>	bootstrap güven aralığı (1000 yeniden örnekleme, tohum 42), eşleşmiş McNemar
<code>eval/ablation.py</code>	dört kol: kural / Ilm / hibrit / hibrit-verify; eşleştirme birimi <code>(belge, alan)</code>
<code>eval/iaa.py</code>	Cohen κ , Fleiss κ , Krippendorff α
<code>eval/properties.py</code>	metamorfik değişmez denetimi

Tüm ölçüm yolu yalnız Python standart kütüphanesiyle koşar. numpy, scipy ve sklearn bağımlılığı yok.

Üç hata sınıfı ayrı ölçülür

Tek bir sayıya bakıp "model kötü" demek yerine hangi hatanın yapıldığını ayırıyoruz; üçü farklı düzeltme gerektiriyor.

- **Kaçırma** — bilgi metinde var, model değer üretmedi. Kapsama sorunu.
- **Yanlış çıkarım** (`fp_wrong`) — bilgi metinde var, model yanlış yerden aldı. Zeminleme sorunu; halüsinasyon sayılmıyor.
- **Halüsinasyon** (`fp_hallucinated`) — bilgi metinde yok, model uydurdu. En tehlikelisi.

Paydalar ayrı. Çıkarım hatası oranının paydası gold'da değer olan kararlar, halüsinasyon oranının paydası gold'un "yok" dediği kararlar. Aynı paydaya bölünürse iki sayı karşılaştırılmaz hâle gelir.

`absent_fields` ayrımı

Klasik gold biçimi "gerçekten yok" ile "anote edilmedi"yi ayırt edemez. Bu ayrım olmadan kesinlik tanımsız kalır: sistem bir değer üretti ve gold'da o alan yoksa, bu yanlış pozitif mi, yoksa anotatör oraya bakmadı mı?

`gold_schema.py` iki liste tutuyor. `fields` anote edilmiş ve değeri olan alanları, `absent_fields` ise anotatörün kontrol edip yokluğunu onayladığı alanları taşıyor. Sistem bir değer üretti ve o alan `absent_fields` içindeyse sonuç kesin halüsinasyon ve sayılabilir.

Neden bootstrap ve McNemar

"F1 = 0,92" bir iddia. 48 kayıtlık bir sette bu sayının belirsizliği $\pm 0,07$ mertebesinde olabilir; iki yaklaşımın farkı bu belirsizlikten küçükse fark yok. Bootstrap güven aralığı sayının ne kadar güvenilir olduğunu söyler. Eşleşmiş McNemar iki kolun farkını sınar; eşleşme kritik, çünkü aynı `(belge, alan)` çiftinde iki kol karşılaştırılıyor ve bağımsız örneklem varsayımı yapılmıyor.

Güven aralığı belge düzeyinde küme bootstrap ile hesaplanıyor. Aynı belgeden çıkan 12 alan bağımsız olmadığı için alan düzeyinde örneklemek aralığı yapay biçimde daraltır.

Ölçülen sonuç

`gold.v2` (n=48, kör etiketlenmiş), `strict` eşleştirici, belge düzeyi bootstrap 1000 örnek, tohum 42. Artefakt: `eval/reports/20260823-073019/`, gold sha `e38a5276...`, commit `f6e76a08`, temiz ağaç.

ölçüt	değer
yapılandırılmış alan mikro-F1 (11 alan)	0,823
12-alan mikro-F1, ikili	0,570 · %95 GA [0,492–0,632]
kalem düzeyi mikro-F1	0,629
makro-F1	0,765
zor dilim (40 belge) mikro-F1	0,587
zor + yapılandırılmış	0,837
halüsinasyon oranı	0,034

Hedef tutulmadı. İkili 12-alan mikro-F1 0,570, ilan edilen 0,60 hedefinin altında. Sayı iki turda 0,4771'den 0,5702'ye çıktı ve orada kaldı. Hedefi sonradan indirmek yerine tutulmadığını kaydediyoruz.

ALAN BAZINDA

alan	ikili F1	not
<code>vade_ay</code>	1,000	
<code>kar_payi_orani</code>	1,000	destek 3, F1 yorumlanamaz
<code>finansman_tutari</code>	1,000	destek 4
<code>kampanya_suresi</code>	0,936	
<code>alisveris_puani</code>	0,933	
<code>taksit_sayisi</code>	0,909	
<code>odul_miktari</code>	0,727	
<code>masraf_durumu</code>	0,667	
<code>indirim_orani</code>	0,667	destek 2
<code>hedef_kitle</code>	0,571	kalem düzeyinde 0,632
<code>kampanya_kosullari</code>	0,000	kalem düzeyinde 0,520
<code>tahsis_ucreti</code>	—	destek 0, tanımsız

KAMPANYA_KOSULLARI NEDEN 0,000

İkili ölçüt tam küme eşitliği arıyor. Bir belgenin koşul listesi birebir eşleşmezse, kaç koşulun doğru çıkarıldığına bakılmadan sonuç $TP=0$, $FP=1$, $FN=1$ oluyor. Bu koşulunda 137 kalemin 78'i doğru çıkarıldı (kalem $P\ 0,479 \cdot R\ 0,569 \cdot F1\ 0,520$) ama hiçbir kayıt birebir küme eşleşmesi vermedi.

Serbest metin listesi döndüren bir alanda anlamlı ölçüt kalem düzeyi. İkili sayı yine yayımlanıyor; onu saklamak ölçütün zayıflığını değil sonucu saklamak olurdu. Üç görünüm birlikte duruyor ve hiçbir diğeri yerine geçmiyor. Yapılandırılmış kesit (11 alan) bu alanı dışlıyor, 0,823 sayısı oradan geliyor.

Gold setin statüsü

set	n	kim etiketledi	hakemlik
gold.v1	20	insan anotatör	geçti, 2 kayıt düzeltildi
gold.v2	48	makine anotatör (M1–M4), her belge birebir alıntı kanıtıyla	yok
gold.round1	134	makine anotatör (A–D), protokol v2	makine kör hakem, 38 kayıt

Manşet 0,570 sayısı gold.v2 üzerinde ölçüldü, yani insan hakemliğinden geçmemiş bir sette. Alternatifi gold.v1 'in 0,677'sini manşete koymak olurdu ve o da modele çapalı bir protokolden geliyor. İkisi de kısıtlı, ikisi de kısıtıyla birlikte sunuluyor.

gold.round1 'deki hakemlik makine hakemliği. 41 uyuşmazlık, yalnız kendi alanının kılavuz paragrafını gören ve birbirinden habersiz çalışan kör hakemlerce karara bağlandı; 18 hücre şema onarımından geçti. Protokol, hakemin A ya da B ile hem karar hem değer olarak örtüşmesini şart koşuyor. Yine de insan hakemliğinin yerine geçmiyor: adjudicated: true bayrağı "hakemlikten geçti" diyor, "insan onayladı" demiyor. **Kapatılması gereken en öncelikli açık bu.**

Senaryonun kalp alanı yeterince ölçülmedi. kar_payi_orani gold.v2 'de yalnız üç karar destekli. Oradan çıkan F1 yorumlanamaz; üç karar bir F1 taşımaz. Alan korpusta 164/2.708 belgede (%6,1) geçiyor ve bu bir model kısıtı olarak okunmamalı — bankalar oranları kampanya sayfalarında büyük ölçüde yayımlanıyor.

Halüsinasyon oranı: iki set doğrudan karşılaştırılmaz

	gold.v2 (n=48)	gold.round1 (n=134)
halüsinasyon oranı	0,034 (15/447)	0,284 (21/74)
payda (absent_decisions)	447	74
skipped_undecided	0	1.380

gold.round1 'in paydası küçük çünkü anotatörler 1.380 alan-kararında hiç karar vermemiş; bunlar metriğe girmiyor. Küçük paydada tek kayıt oranın büyük bir dilimini taşıyor: 21 halüsinasyonun 10'u tek başına vade_ay alanından geliyor. İki oran ayrı ayrı doğru ölçüldü, ama yan yana konup "model round1'de kötüleşti" denemez. Payda küçülmesi bir gold kapsama yoğunluğu artefaktı.

Anotatörler arası uyum

tur	ölçüt	değer	not
v2	Cohen κ	0,714	16 kayıt, 192 çift, ikinci etiketleyici LLM
round0	Fleiss κ	0,302	260 ortak satır, 4 anotatör, hakemlik sonrası
round1	Cohen κ	0,274	141 ortak karar, hakemlik öncesi

Üçü de kabul eşiğinin ($\kappa \geq 0,80$) altında. İlan edilen sonuç uygulandı: `masraf_durumu` alanının negatif κ 'sı hakemlendi, gold ve kılavuz ile birlikte çıkarım motoru düzeltildi.

Değişmez (metamorfik) denetimi

Gold set olmadan doğruluk nasıl denetlenir? Dört değişmez, korpusun tamamında koşuyor:

- **Ortografik değişmezlik** — `çıkar(metin) == çıkar(BÜYÜK(metin))`
- **Boşluk değişmezliği** — fazladan boşluk sonucu değiştirmiyor
- **Konum tutarlılığı** — `metin[span_start:span_end]` gerçekten `raw_value` içeriyor
- **Kanonik tip tutarlılığı** — her alan şemasının tanımladığı tipi döndürüyor

Ölçüm 2026-08-21: **2.708 belgede 0 ihlal**, kapsam %92,3 (2.499 belgede en az bir alan çıktı; 209 boş belgede denetim hiçbir şey sınıyor). Komut `python -m eval.properties --raw-dir data/raw`, çıkış kodu 0.

Denetleyicinin kendisi de sınıyor. `tr_fold()` düzeltmesi geri alınınca denetim gerçekten ihlal üretiyor; ilk koşuda bu sınıftan 104 ihlal bulmuştu. 21 Ağustos'ta iki gerçek ihlal verip CI'ı kırdı (P11).

Bölünmüş test kümesi disiplini

`scripts/split_gold.py` dev/test bölmesi üretiyor ve test bölmesini donduruyor. `--verify` bölmenin değişmediğini hash ile doğruluyor, `--record-access "gerekçe"` her erişimi kayda geçiriyor. Test setine bakıp modeli ona göre ayarlamak ölçümü değersiz kılar; erişim kaydı bu disiplini denetlenebilir yapıyor.

Test paketi

koşucu	toplana	geçti	atlandı	başarısız
<code>unittest</code> (kanonik)	3.948	3.895	53	0
<code>pytest</code> (çapraz doğrulama)	3.948	3.895	53	0

Artefakt: `eval/reports/test-ozeti.json`, commit `03822c24`, `git_dirty: false`, Python 3.14.6. Atlanan 53 test Postgres/pgvector istiyor ve CI'ın `test-with-deps` işinde koşuyor. Panel tarafında 224 arayüz testi (52 küme, 9 dosya).

Hız ve kaynak

Yol	n	p50	p95	p99	maks
kural	5.088	1,03 ms	4,80	6,30	37,63
boru hattı, LLM kapalı	5.088	1,50 ms	6,92	8,86	75,32
sohbet	504	12,48 ms	325,02	351,36	368,53

Verim 21.087 belge/dakika, tepe RSS 100,4 MB. Bu ölçüm 1.696 belgelik korpusta yapıldı; korpus 2.708'e çıktıktan sonra tekrarlanmadı. Sohbet p95/p99 yayılımı 26 kat ve kayıtlı bir performans borcu: RAG kolu tüm gövdede tarama yapıyor.

Konteyner ile host ölçümü neredeyse aynı (kural 1,05 / boru hattı 1,67 ms), yani konteynerleştirmenin gecikme cezası pratikte yok.

Güvenlik değerlendirmesi

İstem enjeksiyonu seti: 22 saldırının 22'si savuşturuldu, 4 kontrol sorusunun 4'ü doğru yanıtlandı. Hem kapılar tek başınayken hem RAG sentezi açıkken aynı sonuç. Kısıt: set n=26 ve sentez tarafı tek modelle (`qwen2.5:7b-instruct`) ölçüldü. Ayrıntı: [docs/rapor/guvenlik-llm-modu.md](#) .

Kanıt tazeliği kapısı

`python -m scripts.kanıt_tazeligi` yayımlanan her sayıyı üreten artefaktla karşılaştırıyor. Sapma CI'ı kırıyor. Son koşum: **16 iddia, 0 sapma** (2026-08-21). Bayat bir manşet sayı belgede kaldığında kapı onu buluyor.

Ölçülmeyen ve açık kalanlar

- İnsan hakemliğinden geçmiş büyük gold set. En öncelikli açık.
- Çelişki tespitinin yanlış negatif oranı.
- Güven skorlarının kalibrasyonu.
- Gecikme ölçümünün 2.708 belgelik korpusta tekrarı.
- `kar_payi_orani` alanı için anlamlı destekli ölçüm; alan korpusta seyrek olduğu için gold örnekleme de seyrek kalıyor.
- Bağımlılık envanteri bir kesit, sözleşme değil. Python kilit dosyası yok (`requirements.txt` `>=` pinleri taşıyor); ortama yeni paket girdiği anda sapma tekrar oluşabilir. Bunu `make lisans-kapisi` ve kanıt tazeliği kapısı yakalıyor.

Kaynaklar

Konu	Belge
Şartname madde madde uyum matrisi	<code>docs/SARTNAME-UYUM.md</code>
Ablasyon ve McNemar ayrıntısı	<code>docs/rapor/ablasyon.md</code>
Offline / on-prem kanıt paketi	<code>docs/OFFLINE-KANIT.md</code>
Model lisans zinciri	<code>docs/model-license-audit.md</code> , <code>docs/LISANSLAR.md</code>
Güvenlik katmanı	<code>docs/rapor/guvenlik-llm-modu.md</code>
Mimari kararlar	<code>../decisions/</code> (16 atomik karar sayfası)
Problem kayıtları	<code>../sorun/</code>
Eşik düşürme disiplini	<code>docs/adr/0001-esik-dusurme-disiplini.md</code>